

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
df = pd.read_csv("/train.csv")

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	

Next steps:

Generate code with df

New interactive sheet

```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

```
# Select features and target

X = df[["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]]
y = df["Survived"]

X.head()
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	
0	3	male	22.0	1	0	7.2500	S	
1	1	female	38.0	1	0	71.2833	C	
2	3	female	26.0	0	0	7.9250	S	
3	1	female	35.0	1	0	53.1000	S	
4	3	male	35.0	0	0	8.0500	S	

Next steps:

Generate code with X

New interactive sheet

```
# Handle missing values

X["Age"] = X["Age"].fillna(X["Age"].median())
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])

# Convert Sex into numerical values
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})

# Convert Embarked into numerical columns
X = pd.get_dummies(X, columns=["Embarked"], drop_first=True)

X.head()
```

/tmp/ipykernel_1921/2267927356.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver

```
X["Age"] = X["Age"].fillna(X["Age"].median())
/tmp/ipykernel_1921/2267927356.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver

```
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])
/tmp/ipykernel_1921/2267927356.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver

```
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S
0	3	0	22.0	1	0	7.2500	False	True
1	1	1	38.0	1	0	71.2833	False	False
2	3	1	26.0	0	0	7.9250	False	True
3	1	1	35.0	1	0	53.1000	False	True
4	3	0	35.0	0	0	8.0500	False	True

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
# Split the dataset into training and testing data

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (712, 8)
Testing data: (179, 8)

```
# Create the Logistic Regression model

model = LogisticRegression(max_iter=1000)

# Train the model
model.fit(X_train, y_train)

print("Model training completed!")
```

Model training completed!

```
# Make predictions

y_pred = model.predict(X_test)

print("Predictions:")
print(y_pred[:20])
```

```
Predictions:
[0 0 0 1 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0]
```

```
# Calculate model accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

Accuracy: 0.8100558659217877
```

```
# Get prediction probabilities

y_prob = model.predict_proba(X_test)

print("Prediction probabilities for the first 5 passengers:")
print(y_prob[:5])

Prediction probabilities for the first 5 passengers:
[[0.88990454 0.11009546]
 [0.77787317 0.22212683]
 [0.8654933  0.1345067 ]
 [0.11381509 0.88618491]
 [0.26366234 0.73633766]]
```

```
# Compare actual and predicted values

comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

comparison.head(10)
```

	Actual	Predicted	
0	1	0	
1	0	0	
2	0	0	
3	1	1	
4	1	1	
5	1	1	
6	1	1	
7	0	0	
8	1	1	
9	1	1	

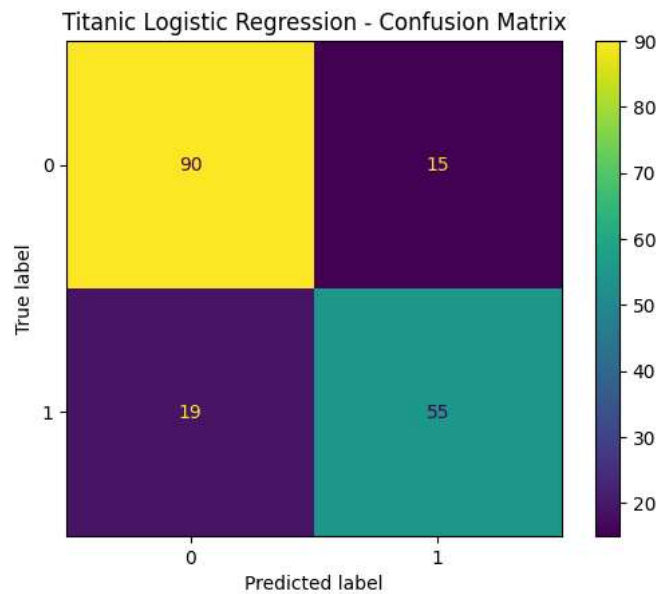
Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)

plt.title("Titanic Logistic Regression - Confusion Matrix")
plt.show()
```



```
print("Titanic Logistic Regression Results")
print("-----")
print("Model Accuracy:", round(accuracy * 100, 2), "%")
```

Titanic Logistic Regression Results

Model Accuracy: 81.01 %